P1018R13
C++ Language Evolution status 🦠 pandemic edition 🦠 2021/08–2021/09

Published Proposal, 2021-09-06

This version:
http://wg21.link/P1018r13

Author:
JF Bastien (Woven Planet)

Audience:
WG21, EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/jfbastien/papers/blob/master/source/P1018r13.bs

Abstract

This paper is a collection of items that the C++ Language Evolution group has worked on in the latest meeting, their status, and plans for the future.

Table of Contents

References

Informative References

§ 1. Executive summary

We have not met in-person since the February 2020 meeting in Prague because of the global pandemic. We're instead holding weekly teleconferences, as detailed in [P2145R1]. We focus on providing non-final guidance, and will use electronic straw polls as detailed in [P2195R0] to move papers and issues forward in a asynchronous manner.

Our main achievements have been:

- **Issue processing**: most of the 50 language evolution issues have proposed resolutions, and a large number of them have been voted on through electronic polling.
- **C++23**: we've started work on papers for C++23 and later.
- **Incubation**: we've acted as EWG-I and "incubated" some early papers by providing early feedback to authors.

This paper outlines:

- The work achieved in § 7 Teleconferences;
- Lists the § 5 Poll for the August 2021 polling period and explain the § 6 Polling Process (see [P1018r9] for the results of the February 2021 polling period and [P1018r11] for the results of the May 2021 polling period);
- Lists the remaining outstanding issues in § 8 Remaining Open Issues, some are still open while others are waiting for polling;
- § 9 Near-future EWG plans.

§ 2. Paper of note

- [P1000R4] C++ IS schedule
- [P0592R4] To boldly suggest an overall plan for C++23
- [P1999R0] Process: double-check evolutionary material via a Tentatively Ready status
- [P2195R0] Electronic Straw Polls
- [P2145R1] Evolving C++ Remotely

§ 3. Tentatively ready papers

Following our process in [P1999R0], we usually mark papers as tentatively ready for CWG. We would usually take a brief look at the next meeting, and if nothing particular concerns anyone, send them to CWG. However, given the pandemic, we've decided to provide guidance only in virtual teleconferences, and have an asynchronous polling mechanism to officially send papers to CWG or other groups as detailed in [P2195R0].

You can follow the lists of papers on GitHub:

- tentatively ready papers,
- EWG vote on me papers.

§ 4. Issue Processing

We've reviewed 50 Language Evolution issues at the Core groups' request, and have tentative resolutions for most. We don't want to poll all of these at the same time, and therefore only poll a subset in each polling period, reserving other issues for later polling periods. We've therefore only poll the "tentatively ready" issues (since they're tied to papers, and polled with said papers, as outlined below), as well as the "resolved" issues since telecon attendees believe that prior work has already addressed the issues.

§ 8 Remaining Open Issues contains a list of issues which aren't being voted on in this polling period.

§ 5. Poll

The results of the polls which EWG took in the August 2021 polling period are:

§ 5.1. P2138r4 Rules of Design <=> Wording engagement

- [P2138r4]
- GitHub issue
- **Highlight:** process description of how proposal reviews should work between Language Evolution and Core, as well as between Library Evolution and Library.
- **Poll:** For the next plenary, put up the following joint Language and Library Evolution motion: P2138r4 "Rules of Design <=> Wording engagement" as the official process of the C++ Evolution groups.

Poll votes:

SF	F	N	A	SA
6	15	3	3	6

Poll outcome: ❌ no consensus. LEWG electronic poll results are similarly distributed.

Salient comments:

- **Strongly favor:** This is the golden standard for process and a good description of the current process.
- **Strongly favor:** Formalizing the process we've agreed to will make it easier to object when we try to rush.
- **Strongly favor:** We need a checklist and a description for how proposals flow through the committee; it's unreasonable to continue to do that in an ad-hoc manner.
- **Neutral:** Not sure evolving processes during covid time is a good idea
- **Neutral:**I'm torn on this paper. It gives a clear description of the process of digesting a proposal and the rationale behind it. I'm all for clear process descriptions - there's even an ISO norm for that. But I can't see how it helps adressing bullet point 1 in the list of problems. The scarce ressource of "wordsmiths" is unavoidably tapped in the design cycle (!), effectively sucking the people involved there into a gatekeeping function in the design process, asking them for doing work outside of their group, and possibly eventually stalling the process there. In other words, the former specification review -> design feedback cycle is now embedded into the new design cycle. Looking at the whole process description I can see an almost certain slow-down of the pipeline, an improved quality of the process itself, and the hope for higher quality proposals forwarded to plenary. Unfortunately, improving process quality isn't correlated with improvements in quality of the processed items.
- **Against:** The problems this process aims to address are plainly real: lots of our limited wording-group bandwidth is consumed by reviewing wording that is unusably incomplete, and disagreements about the design intent of papers approved by design groups have been very problematic. Also, distinguishing papers with approved designs but no wording is simply good process engineering, since they can't be reviewed yet. However, it is not clear that the design groups can reliably evaluate whether wording implements the design intent without duplicating the wording group's work. Also, wordsmiths' time is also limited, and while there's no requirement that they agree with a proposal to review its initial wording, the natural tendency would be to spend their limited time on those that they favor. It doesn't seem right to subject proposals to that sort of pocket veto. A wording review group would of course be well within its rights to suspect that a paper's wording might not be ready and deputize one of its members to ascertain that fact and communicate it to the author (who would be just as capable of securing help with it as they would be under this proposal). While the paper discusses means of accelerating urgent proposals, it is nonetheless plain that this process would in general extend the review process. That's not intrinsically undesirable, but lengthening the process by fiat (rather than, say, because implementation concerns have been raised) is of questionable value. It's a little unreasonable to require a notional author of one paper to attend every in-person meeting (when those are happening) for years in case one of the many reviews for it occurs at one of them. In particular, having a "park for further design feedback" stage in addition to a "Tentatively Wording" stage seems gratuitous.

- **Against:** I'm not a fan of the requirement that wording must exist to advance out of (L)EWG. My experience suggests that this actively causes harm. What we've seen is that authors now see that they need wording, so they spend time writing wording. The problem is, the wording is often bad anyway -- so you end up with a paper with potentially pages of wording that doesn't really help explain the design because the authors spent time writing bad wording that could have been spent writing clearer motivation, examples, prose, etc. Or the author knows that they can't write wording, so they go try to get wording review for a paper that may just not be a good design, which ends up not being a good use of the wordsmith's time either. Even when the wording is fine, it's just actively distracting from the discussion - which should be at this point about the design. So you get comments about wording issues that are, frankly, completely irrelevant and wholly unproductive. I think the best thing you could say about requiring wording is that it's simply a waste of time in that the author had to spend their time producing wording earlier than necessary. The presented problem (papers show up to CWG/LWG with poor wording) can be solved by just requiring that papers show up to CWG/LWG with good wording - requiring EWG/LEWG to see it seems like a completely needless extra step in the process.
- **Against:** The proposed approach further complicates already complicated committee workflow without solving the real issue in the committee. People who used to ignore the rules will continue ignore the rules.
- **Abstain:** I don't care. The proposal would change nothing in practice
- **Strongly against:** This paper: does not make any distinction between tiny improvements and huge feature changes; does not acknowledge the need for quicker turn-around on National Body comments during the Committe Draft review stage; incorrectly assumes that the perceived quality issues with recent proposals relates to a lack of formalized process rather than a failure on the part of experienced WG21 participants to actively help to improve proposals; introduces unhelpful gatekeeping and obscured hurdles for new WG21 participants to jump through. Not convinced this will improve processes and am concerned it will cause contributor burnout. This proposal will have the following effects if adopted by LEWG: 1. It will cement a complex and baroque process even further into WG21. 2. It will increase the already high gatekeeper discrimination of proposals. 3. It will allow LWG to discriminate against LEWG by discounting their consensus findings. 4. It will place unbounded decision power on individuals with undefined credentials. Hence it is unconscionable of me to vote anything but strongly against.
- **Strongly against:** We should not optimize our process against small papers or against new participants.
- **Strongly against:** This paper bloats the process with unnecessary checks and abdicates the responsibility of actually vetting the design from the initial design groups to people later in the stage. The time to check for deployment experience and implementation experience is in the incubator and design groups, not before plenary when things are ALMOST ready to head out the door. The proposal also adds mandatory waiting times, rather than leaving it discretionary-by-default, and requires chairs and other individuals to lodge special exceptions in more cases (not less) to move a paper forward. Nothing about this paper improves the process in a material or tangible way that benefits the proposal process, and all it does is add in more places for a proposal to fail and be sent all the way back to the beginning of the process. As a person who ahd to carry a few proposals that suffered from that, this paper is perhaps the worst possible path forward I can imagine for the evolution of C++.
- **Strongly against:** This paper will have disastrous consequences on WG21 that the committee, its members, and proposal writers are wholly unprepared to deal with. If the idea is that, instead of an open process, we should endeavor to make improving C++ so difficult that it makes COBOL's submission process look easy or similar by comparison, then I suppose this paper is "working as intended". If this paper moves through to plenary and is adopted I can see several people simply not participating in C++ anymore. But then again, making the process less accessible may be the point?

§ 5.2. P2266r1 Simpler implicit move

- [P2266r1]
- GitHub issue
- **Highlight:** In C++20, `return` statements can *implicitly move* from local variables of *rvalue* reference type; but a defect in the wording means that *implicit move* fails to apply to functions that return references. C++20's implicit move is specified via a complicated process involving two overload resolutions, which is hard to implement, causing implementation divergence. We fix the defect and simplify the spec by saying that a returned move-eligible *id-expression* is always an *xvalue*.
- **Poll:** Forward P2266r1 "Simpler implicit move" to Core for C++23.

Poll votes:

SF	F	N	A	SA
11	14	5	2	2

Poll outcome: ✅ consensus. However, the against votes are from implementors, and bring relevant new information. This topic needs to be re-discussed, and might fail a plenary vote.

Salient comments:

- **Strongly Favor:** This is a bugfix that unifies handling. Should have been the case from the beginning.
- **Strongly Favor:** This makes the language much more pleasant to use, and fixes a decades-old problem of returning references to locals.
- **Strongly Favor:** This is a sensible improvement for the rules on implicit move.
- **Strongly Favor:** Fixing these kind of corner cases makes the language more consistent and easier to use and teach.
- **Strongly Favor:** The great talk on C++Now 2021 explained everything.
- **Favor:** The `decltype(auto)` case is unfortunate but overall it's a simpler rule.
- **Neutral:** I am in favor of most of this proposal, however the interactions with decltype(auto) have me slightly concerned it will cause more surprising behavior. I cannot decide either way.
- **Neutral:** I'm in two minds about this: on the one hand, the proposal seems useful and matches what some users expect, on the other hand, this is adding even more magic to a part of the language that's already full of implicit rules that are increasingly hard to keep in mind, and this will change what some other users expect.
- **Against:** The rule change appears reasonable, but Arthur's implementation did reveal breaks in real code, and the implementation experience has not been discussed in committee. I do not believe this is ready to move to core.
- **Against:** I was in favor of this until we started getting implementation experience with it. The implementation work done in Clang exposed issues in the MSVC STL headers (among others) where code stopped compiling. I don't think this goes far enough to keep working code working, as this turned out to be a disruptive change.
- **Strongly Against:** This has, as far as I understand, not been implemented nor tested. Yet it changes the meaning of currently-valid code. Until we have reasonable evidence that it has been implemented this should not move forward.
- **Strongly Against:** The implementation in clang ended up causing a ton of failures in existing code, particularly in the MSVC headers. We've been dealing with the fallout from the clang-implementation for about a week and a half now, and it is really painful so far.

§ 5.3. P2128r5 Multidimensional subscript operator

- [P2128r5]
- GitHub issue
- **Highlight:** user-defined types can define a subscript operator with multiple arguments to better support multi-dimensional containers and views, for example `template<class... IndexType> constexpr reference operator[](IndexType...);`.
- **Poll:** Forward P2128r5 "Multidimensional subscript operator" to Core for C++23.

Poll votes:

SF	F	N	A	SA
20	11	1	2	1

Poll outcome: ✅ consensus.

Salient comments:

- **Strongly Favor:** The language allows expressing a difference between invocation and indexing for single-element forms but not for multi-element forms; with this proposal, that inconsistency is fixed, and programmers who want to make the difference clear can do so, going forward.
- **Strongly Favor:** This is why we deprecated comma inside the subscript expression. Given the small number of cases where this is used that have so far been reported I see no problem with moving ahead with this for C++23.

- **Strongly Favor:** Avoids hijacking the function call operator for subscription.
- **Strongly Favor:** We would adopt this as soon as possible if/when made available.
- **Favor:** This is natural extension of the syntax, however I am not convinced with repurposing syntax, before making it ill-formed for at least one standard.
- **Favor:** This is one of those questions that comes up again and again, and the status quo is made worse by the existence of comma expressions, which give the illusion that we already support this feature. The comma expression behavior is unintuitive and surprising (and deprecated since C++20 in order to make room for this feature). Let's fix that.
- **Against:** I'm not comfortable with the fact that both a[1, 2, 3] and a(1, 2, 3) might be valid at the same time, but mean totally different things. I would like to see restrictions against a type having both operator() and operator[], or at least a discussion of this issue
- **Against:** I'm not sold on the motivation and value of this change. While true, the question is pretty common, but so is the answer and where it emerged the programmer could pick one of 3 ways, each straightforward. Strange, that the 3 are not the same we see in the paper, the baseline, that is to use a named finxtion is not even mentioned. And the code that uses .At(x, y, z) may have a few extra characters but is simple to read and write. Has less gotchas than the other 2 ways -- using other operator like () or using an indexing object. It also solves the "generic code" problem. Unfortunate that our standard lib did not follow the guidelines of the era and only introduced a .at() function with a different contract. If the language had this, I probably could find a use somewhere but remain skeptic it would take over in general due to the many existing libraries. We'd just have the N+1 problem for long time. And keep writing shims for generics. My gut tells me that if the paper did have the non-member op[] with demonstration that I could use it with the old libs and make my code switch to a single [] syntax, that might switch me to F. But can't tell without looking at all the fallout.
- **Strongly Against:** This doesn't actually add significant capability, but it does add confusion.

§ 5.4. P2036r2 Changing scope for lambda *trailing-return-type*

- [P2036r2]
- GitHub issue
- **Highlight:** the way that name lookup works in lambdas is surprising: it behaves differently in the trailing-return-type than it does in the lambda body. The current behavior is surprising and error-prone. The proposed change treats the *trailing-return-type* like the function body rather than treating it like a function parameter.
- 🗳️ **Poll:** Forward P2036r2 "Changing scope for lambda trailing-return-type" to Core for C++23. Treat it as a Defect Report.

Poll votes:

SF	F	N	A	SA
19	11	0	0	1

Poll outcome: ✅ consensus.

Salient comments:

- **Strongly Favor:** Names in a lambda should be looked up with a consistent set of rules
- **Strongly Favor:** Colleagues expect the behavior this paper proposes rather than the behavior without.
- **Strongly Against:** This has, as far as I understand, not been implemented nor tested. Yet it changes the meaning of currently valid code.

§ 5.5. P2334r1 Add support for preprocessing directives `elifdef` and `elifndef`

- [P2334r1]
- GitHub issue
- **Highlight:** the `#ifdef` and `#ifndef` preprocessing directives exist today as shorthand for `#if defined(identifier)` and `#if !defined(identifier)`, respectively. However, no analogous shorthand preprocessing directives exist for `#elif defined(identifier)` and `#elif !defined(identifier)`. Add two

new preprocessing directives `#elifdef` and `#elifndef` that exactly parallel the functionality supplied in the existing `#ifdef` and `#ifndef` directives.

- **Poll:** Forward P2334r1 "Add support for preprocessing directives `elifdef` and `elifndef`" to Core for C++23.

Poll votes:

SF	F	N	A	SA
11	13	9	3	2

Poll outcome: ✅ consensus.

Salient comments:

- **Strongly Favor:** Compatibility with the C preprocessor is important. I have had colleagues asking why this is missing in previous versions of C/C++
- **Strongly Favor:** Even though not strictly necessary, the gap closed by this proposal looks so obvious in hindsight that I can't find a good reason to object to that.
- **Strongly Favor:** Speaking for myself, I've wanted this feature for nearly as long as I've been programming in C++. Given that C is adopting this change to the preprocessor, I don't see how we could reasonably refuse to adopt it.
- **Strongly Favor:** It fills a hole that people have been complaining about for 40 years; WG14 already adopted the feature and a diverging preprocessor would be Very Bad.
- **Favor:** While tweaks to the preprocessor aren't the best extensions we could spend our time on, this seems reasonable.
- **Favor:** We would prefer the preprocessor be, as closely as possible, the same between C and C++.
- **Favor:** In a vacuum, optimizing conditional compilations when we should try to have less of it is a waste of time. But WG14 is forcing our hand, the work was done, users like it, ship it.
- **Favor:** Compatibility with C is important, changes in that should be adopted unless there is a good reason not to. Here we have no reason. If it was not coming up from C, I'd vote SA as the motivation is moot and the change not necessary for anything.
- **Favor:** C compat.
- **Favor:** This proposal is trivial; it might not be worth the bother, but alignment with C makes it plainly desirable.
- **Favor:** Given WG14's interest in this, too, this seems helpful.
- **Favor:** Keeps C++ aligned with upcoming changes in C. Aligns with user expectations. Seems like there's some good and no harm here at all.
- **Favor:** I'm not thrilled by adding features to the preprocessor, but this seems to be liked by WG14, making this a C23 compatibility issue.
- **Favor:** A simple and useful facility.
- **Favor:** We should minimize preprocessor usage, but this seems straightforward
- **Neutral:** Folks have asked for this for years, but this is just a compatibility thing. I care neither way.
- **Against:** Let us not extend the preprocessor any further. It is okay for PP code to be ugly as hell, this way we will get proper alternatives faster.
- **Against:** It is worrying that on implementations without this feature, uses of these directives can still compile but not do what you want: #if 0 ... #elifdef Y ... #endif is valid today and simply preprocesses to nothing without checking Y.
- **Against:** This does not bring new functionality to the language, does not solve real world problem. It does not worth committee time.
- **Strongly Against:** As far as I understand this has not been implemented or tested. This creates bifurcation in the preprocessor (between C and C++, between new and old), in an area that is specifically used to deal with different compiler versions... but this feature cannot be used that way since it doesn't exist in current and older versions.
- **Strongly Against:** Completely redundant useless feature, adds unnecessary complexity to preprocessor. Adding it to C was a mistake which we shouldn't repeat.

§ 5.6. P2066r7 Suggested draft TS for C++ Extensions for Transaction Memory Light

- [P2066r7]
- See [P1875r2] for discussion.
- Current TS [N4514]: ISO/IEC TS 19841:2015 Technical Specification for C++ Extensions for Transactional Memory.
- GitHub issue
- **Highlight:** transactions can be expressed with atomic statements of the form `atomic do { /* work goes here */ }`.
- **Poll:** Forward P2066r7 "Suggested draft TS for C++ Extensions for Transaction Memory Light" to Core to become the Transactional Memory TS v2.

Poll votes:

SF	F	N	A	SA
6	12	3	0	1

Poll outcome: ✅ consensus.

Salient comments:

- **Strongly Favor:** Looks like good MVP for TS.
- **Strongly Favor:** Our last hope to see transactional memory TS actually implemented.
- **Strongly Favor:** The UK NB is happy to see this. There's useful section "questions for TS deployment" explaining what is hoped for from the TS process.
- **Favor:** This feature helps users writing safer code without the need of understanding all of the possibly non-obvious interactions of atomics. In particular when there's more than one atomic involved.
- **Favor:** This looks like an marked improvement over the first TS, and hopefully will permit gathering some experience with transactional memory use in C++.
- **Favor:** Not sure what the goal here is, but sure. Worth noting that we have a Reflection TS that nobody has implemented.
- **Favor:** Recent reflector discussions seem to have established that this proposal has the correct degree of maturity to become a Technical Specification; the implementation experience one hopes to derive via that process will be valuable.
- **Neutral:** The paper lacks motivation and implementers of compiler and chips do not seem super enthusiastic about it. Only ARM seems to have the hardware support.
- **Strongly Against:** It is a huge mistake to re-use "atomic", which we have already been shipping as a templated type for over a decade. It may limit future design choices. Once this TS is out there, the bar for changing this contextual keyword is much, much higher. This isn't even one of the questions being asked of the TS deployment. A word that isn't quite so common would be far better.

§ 5.7. P2360r0 Extend *init-statement* to allow *alias-declaration*

- [P2360r0]
- GitHub issue
- **Highlight:** This is currently valid `for (typedef int T; T e : v) /* something */;` but this isn't `for (using T = int; T e : v) /* something */;`. Make the later one valid.
- **Poll:** Forward P2360r0 "Extend *init-statement* to allow *alias-declaration*" to Core for C++23.

Poll votes:

SF	F	N	A	SA
13	12	7	2	0

Poll outcome: ✅ consensus.

Salient comments:

- **Strongly Favor:** An obvious fix with no side effects
- **Strongly Favor:** It's surprising that it doesn't work already; removing discrepancies between typedef and using here is a good thing
- **Favor:** Once again, not strictly a necessary feature that's proposed here. But a feature that fixes an inconsistency, that is allegedly easy to implement, and that breathes "modern" is a good thing.
- **Neutral:** On one side I agree that it is an inconsistency and overlook, I see no practical reason for any client to notice the difference. Accepting the change improves the language but I can't evaluate the implementation cost. If it's provably low, then I vote F. If it's adopted it should be a DR to close the incosistency gap for good. And that rises the bar even more. If restricted to just for C++23 onwards I more lean to A.
- **Against:** This paper offers to fix an inconsistency to a grammar that we don't know make sense and the code it allows to write is unlikely to pass many code reviews.
- **Against:** This needs an implementation and testing.

§ 5.8. P2246r1 Character encoding of diagnostic text

- [P2246r1]
- GitHub issue
- **Highlight:** the standard provides a few mechanisms that suggest an implementation issues a diagnostic based on text written in the source code. However, the standard does not uniformly address what should happen if the execution character set of the compiler cannot represent the text in the source character set. Because the display of diagnostic messages should be merely a matter of Quality of Implementation, the proposal is to place no character set related requirements on the diagnostic output with the understanding that implementations will do what makes the most sense for their situation when issuing diagnostics in terms of which characters need to be escaped or otherwise handled in a special way.
- 👍 **Poll:** Forward P2246r1 "Character encoding of diagnostic text" to Core for C++23.

Poll votes:

SF	F	N	A	SA
12	19	1	0	0

Poll outcome: ✅ consensus.

Salient comments:

- **Strongly Favor:** This keeps C++ and C in step, with WG14's N2563 paper having been adopted.
- **Strongly Favor:** One of a number of overdue encoding clarifications/fixes.

§ 5.9. P2314r2 Character sets and encodings

- [P2314r2]
- GitHub issue
- **Highlight:** switch C++ to a modified "model C" approach for universal-character-names as described in the C99 Rationale v5.10, section 5.2.1. Introduce the term "literal encoding". For purposes of the C++ specification, the actual set of characters is not relevant, but the sequence of code units (i.e. the encoding) specified by a given character or string literal are.
- 👍 **Poll:** Forward P2314r2 "Character sets and encodings" to Core for C++23.

Poll votes:

SF	F	N	A	SA
14	12	2	0	0

Poll outcome: ✅ consensus.

Salient comments:

- **Strongly Favor:** Important clean up and clarification that brings the language specification much closer to the way that it's actually implemented.
- **Strongly Favor:** Having the term "Literal encoding" makes life much easier when discussing text.
- **Strongly Favor:** This improves the quality of the standard with a clearly specified terminology that happens to match the behaviour of existing implementations. The most noble objective of the committee is hereby met.
- **Strongly Favor:** This paper is crucial to the work SG16 is currently doing to improve Unicode during lexing and fixes some misconception about how lexing works
- **Neutral:** I'm SF for a paper that resolves 5 core issues for good and really just codifies what is already done by implementations. However, evaluating that it is indeed the case goes beyond my comprehension. The claim about the implementation reads like just being derived from one code sample and behavior of #. Rather than a direct query of implementors on the mechanic itself. The March 10 SG16 poll suggest that even the experts are unsure and their later turning to favor may come from the wrong motivation. I'd sugges caution on this and hope the votes that will count will come only from people who are confident in both the design and the wording.

§ 5.10. P2316r1 Consistent character literal encoding

- [P2316r1]
- GitHub issue
- **Highlight:** character literals in preprocessor conditional are changed to behave the same way as they do in C++ expressions. For example, `'A' == '\x41'` is not currently guaranteed to behave the same in a preprocessor condition as it is in a C++ expression.
- **Poll:** Forward P2316r1 "Consistent character literal encoding" to Core for C++23. Treat it as a Defect Report.

Poll votes:

SF	F	N	A	SA
16	15	0	1	0

Poll outcome: ✅ consensus.

Salient comments:

- **Strongly Favor:** One of the many encoding related fixes that are long overdue.
- **Strongly Favor:** Improving encoding issues is very important. The old behaviour was inconsistent at best, so I think it is reasonable to treat this as a DR
- **Strongly Favor:** The fewer defects with character encoding the ~~better~~
- **Favor:** Consistency is always a laudable goal in a specification. Here it's even in the title of the proposal. And its content matches existing implementations. The only "drawback" is it might render existing, already broken code officially as such.
- **Favor:** This paper proposes to standardize the behavior of existing implementations.
- **Against:** I could write an example very similar to the motivating example with integer literals, so this issue does not look at all discrepancies. By solving one and ignoring the others, it increases the risk that someones does a mistake. Small detail: The paper says: "Because this paper proposes to standardize the behavior of all existing implementations" but only 4 implementations are mentioned...

§ 6. Polling Process

For each poll, participants are asked to vote one of:

- Strongly in favor
- In favor

- Neutral
- Against
- Strongly against

Participants also have the option to abstain from voting on a particular poll. They are asked to comment on each poll. This comment is mandatory, as it helps the chair determine consensus.

§ 7. Teleconferences

Here are the minutes for the virtual discussions that were held since the Prague meeting in February 2020:

1. 2020-04-09 Issue Processing
2. 2020-04-15 Issue Processing
3. 2020-04-23 Issue Processing
4. 2020-04-29 Issue Processing
5. 2020-05-07 Issue Processing
6. 2020-05-13 Issue Processing
7. 2020-05-21 A General Property Customization Mechanism—[P1393R0] (P1393 tracking issue)
8. 2020-06-10 Reviewing Deprecated Facilities of C++20 for C++23—[P2139R1] (P2139 tracking issue)
9. 2020-06-18 C++ Identifier Syntax using Unicode Standard Annex 31—[P1949R4] (P1949 tracking issue)
10. 2020-06-24 Extended floating-point types and standard names—[P1467R4] (P1467 tracking issue)
11. 2020-07-02 Allow Duplicate Attributes, Attributes on Lambda-Expressions—[P2156R0] (P2156 tracking issue) and [P2173R0] (P2173 tracking issue)
12. 2020-07-08 Pointer lifetime-end zap and provenance, too—[P1726R3] (P1726 tracking issue)
13. 2020-07-16 Guaranteed copy elision for return variables—[P2025R1] (P2025 tracking issue)
14. 2020-07-30 Reviewing Deprecated Facilities of C++20 for C++23, Removing Garbage Collection Support—[P2139R2] (P2139 tracking issue) and [P2186R0] (P2186 tracking issue)
15. 2020-08-05 Transactional Memory Lite Support in C++—[P1875R0] (P1875 tracking issue)
16. 2020-08-19 Freestanding Language: Optional `::operator new`, Mixed string literal concatenation—[P2013R2] (P2013 tracking issue) and [P2201R0] (P2201 tracking issue)
17. 2020-08-27 Exhaustiveness Checking for Pattern Matching—[P1371R3] (P1371 tracking issue)
18. 2020-09-02 `#embed` - a simple, scannable preprocessor-based resource acquisition method—[P1967R2] (P1967 tracking issue)
19. 2020-09-10 A pipeline-rewrite operator—[P2011R1] (P2011 tracking issue)
20. 2020-09-16 Pattern matching: inspect is always an expression—[P1371R3] (P1371 tracking issue)
21. 2020-09-24 C++ Identifier Syntax using Unicode Standard Annex 31, Member Templates for Local Classes—[P1949R6] (P1949 tracking issue) and [P2044R0] (P2044 tracking issue)
22. 2020-09-30 Narrowing contextual conversions to bool, Generalized pack declaration and usage—[P1401R3] (P1401 tracking issue) and [P1858R2] (P1858 tracking issue)
23. 2020-10-08 Compound Literals, `if consteval`—[P2174R0] (P2174 tracking issue) and [P1938R1] (P1938 tracking issue)
24. 2020-10-14 Inline Namespaces: Fragility Bites, Trimming whitespaces before line splicing—[P1701R1] (P1701 tracking issue) and [P2223R0] (P2223 tracking issue)
25. 2020-10-22 Issues Processing
26. 2020-10-28 Issues Processing
27. 2020-11-05 Deducing `this`—P0847R5 (P0847 tracking issue)
28. 2020-11-19 `goto` in pattern matching
29. 2020-12-03 `auto(x)`: decay-copy in the language—P0849R5 (P0849 tracking issue)

30. 2021-01-14 Polls
31. 2021-01-20 Final polls preparation
32. 2021-01-28 P2012 Fix the range-based for loop
33. 2021-02-03 P2280 Using unknown references in constant expressions
34. 2021-02-11 P1974 Non-transient constexpr allocation using `propconst`
35. 2021-02-17 P2242 Non-literal variables (and labels and `goto`s) in `constexpr` functions
36. 2021-02-25 P1371 pattern matching: implementation experience pattern matching now has fairly capable prototype implementation and in this session it will be both demonstrated and described. Bruno, the core implementer, will also field any implementation related questions surrounding the proposal. See the Godbolt demo.
37. 2021-03-02 P2279R0 We need a language mechanism for customization points joint EWG+LEWG session
38. 2021-03-11 P2285 Are default function arguments in the immediate context?
39. 2021-03-17 P2266 Simpler implicit move
40. 2021-03-25 P2128 Multidimensional subscript operator
41. 2021-03-31 P2036 Changing scope for lambda trailing-return-type, and P2287 Designated-initializers for base classes
42. 2021-04-08 Pattern matching identifier patterns syntax: `let` vs `auto` vs none Introducing bindings inside pattern matching: Unqualified identifier within pattern matching must have well defined meaning. Published version of the paper (P1371R4) proposed using case keyword to disambiguate names between identifier expressions and introducing named binding. This direction raised some concerns within committee and authors want to propose another way of resolving the ambiguity. After the presentation authors would like to have a poll to secure committee approval on the proposed direction.
43. 2021-04-22 P2334 Add support for preprocessing directives `elifdef` and `elifndef`, and P2246 Character encoding of diagnostic text
44. 2021-04-28 Final discussion of the polls, and P2066 Suggested draft TS for C++ Extensions for Minimal Transactional Memory
45. 2021-05-06 P2314 Character sets and encodings, and quick review of r2 for P2280 Using unknown references in constant expressions
46. 2021-05-12 P2255 A type trait to detect reference binding to temporary
47. 2021-05-20 P2025 Guaranteed copy elision for named return objects
48. 2021-05-26 P2041 Deleting variable templates
49. 2021-06-03 P1967 `#embed` — a simple, scannable preprocessor-based resource acquisition method
50. 2021-06-14 Joint session with LEWG on P2138 Rules of Design `<=>` Specification engagement
51. 2021-06-23 P1701 Inline Namespaces: Fragility Bites
52. 2021-07-01 P2360R0 Extend init-statement to allow alias-declaration, and P2290 Delimited escape sequences, and P2316 Consistent character literal encoding
53. 2021-07-07 P2392R0 Pattern matching using `is` and `as`
54. 2021-07-21 P2350 constexpr class
55. 2021-07-29 P1169 `static operator()`
56. 2021-08-04 P2347 Argument type deduction for non-trailing parameter packs
57. 2021-08-12 P2355 Postfix fold expressions
58. 2021-08-26 P2295 Support for UTF-8 as a portable source file encoding and P2362 Remove non-encodable wide character literals and multicharacter wide character literals
59. 2021-09-01 P2414R1 Pointer lifetime-end zap proposed solutions

§ 8. Remaining Open Issues

The following table lists all remaining open issues referred to EWG by Core or Library. Some of them are ready to be polled but are held back from the polling periods to limit the number of polls in this round.

From	#	Title	Notes

Core	[CWG2261]	Explicit instantiation of in-class `friend` definition	`struct S {`
			`  template <class T> friend void f(T) { }`
			`};`
			`template void f(int);  // Well-formed?`
			
			A `friend` is not found by ordinary name lookup until it is explicitly declared in the containing name but declaration matching does not use ordinary name lookup. There is implementation divergence on handling of this example.
			
			Note 2020-04-29 Tentative agreement: This should be well-formed.
			
			SF 1 F 10 N 2 A 1 SA 0
			
			JF emailed EWG / Core about this.
			
			Davis: the current name lookup approach which Core is taking in p1787 would disallow this. Support is possible, it would be inconsistent, but would also be a feature.
			
			Notes 2020-10-22: wait until p1787 is voted into the working draft, because it's making this behavior intentional. At that point, we can vote on marking the issue as Not a Defect. No objection to unanimo consent.

Core	[CWG2270]	Non-`inline` functions and explicit instantiation declarations	[Detailed description pending.]
			
			Hubert: question over the role of the `inline` keyword in relation to explicit instantiation declaratio
			
			For `inline` functions, explicit instantiation declarations do not have the effect of suppressing implic instantiation.
			
			A user's desire for wanting to suppress implicit instantiation can arise for different reasons:
			
			To reduce space in object files, executables, etc. and similarly to reduce the number of input symbols linker
			
			To reduce compile time in performing semantic analysis for instantiations whose definitions are provi elsewhere
			
			To control point-of-instantiation, avoiding contexts where the requisite declarations are not declared
			
			The special rule around inline functions allows `inline`-ness to be used to indicate that the first reasc intent and that instantiation-for-inlining is okay.
			
			Consider the following as a translation unit:
			
			`template <typename T>`
			`//inline`
			`void f(T t) { g(t); }`
			`enum E : int;`
			`extern template void f(E);`
			`void h(E e) { f(e); }`
			
			Marking the template definition `inline` would mean that the intended declaration for `g` would need provided as the best candidate at the points-of-instantiation for `f<E>`.
			
			The issue initially points out that this use of the `inline` keyword does not match a view that `inlin` essentially an ODR tool to allow multiple definitions (as opposed to a way to indicate desire for inlini proposed that `extern template` merely has the effect of suppressing definitions in terms of linkag

			(regardless of the `inline` keyword). Such a change would affect the usability of the feature for user that falls within the latter two options above. I am not sure if CWG is asking EWG a specific question other than the general "we do not believe thi wording or obvious consistency issue; is this an issue in terms of design?" **Meeting:** Hubert forked the thread on the reflector. Might want the education SG to take a look, or might want **Meeting 2020-10-28**: Inbal will try to put together the wording, to prove / disprove whether this is a c

Lib	[LWG2432]	initializer_list assignability	std::initializer_list::operator= [support.initlist] is horribly broken and it needs deprecation:
			
			std::initializer_list<foo> a = {{1}, {2}, {3}};
			
			a = {{4}, {5}, {6}};
			
			// New sequence is already destroyed.
			
			Assignability of initializer_list isn't explicitly specified, but most implementations supply a default assignment operator. I'm not sure what [description] says, but it probably doesn't matter.
			
			Proposed resolution:
			
			Edit [support.initlist] p1, class template initializer_list synopsis, as indicated:
			
			namespace std {
			template<class E> class initializer_list {
			public:
			[…]
			constexpr initializer_list() noexcept;
			initializer_list(const initializer_list&) = default;
			initializer_list(initializer_list&&) = default;
			initializer_list& operator=(const initializer_list&) = delete;
			initializer_list& operator=(initializer_list&&) = delete;
			constexpr size_t size() const noexcept;
			[…]
			};
			[…]
			}

			LWG telecon appears to want a language change to disallow assigning a braced-init-list to an std::initializer_list but still permit move assignment of std::initializer_list objects. That is,
			
			auto il1 = {1,2,3};
			
			auto il2 = {4,5,6};
			
			il1 = {7,8,9}; // currently well-formed but dangles immediately; should be ill-formed
			
			il1 = std::move(il2); // currently well-formed and should remain so
			
			Meeting: Proposed resolution:
			
			`initializer_list(const initializer_list&) = default;`
			`initializer_list(initializer_list&&) = default;`
			`[[deprecated]] initializer_list& operator=(const initializer_list&) = def`
			`[[deprecated]] initializer_list& operator=(initializer_list&&) = default;`
			
			SF F N A SA
			
			0 3 12 0 0
			
			JF emailed LEWG, to see if they have an opinion, no feedback. Asked LEWG chairs to schedule for a telecon.
			
			LWG discussed priority.

Lib	[LWG2813]	std::function should not return dangling references	If a std::function has a reference as a return type, and that reference binds to a prvalue returned by the that it wraps, then the reference is always dangling. Because any use of such a reference results in und behaviour, the std::function should not be allowed to be initialized with such a callable. Instead, the p should be ill-formed.
			
			A minimal example of well-formed code under the current standard that exhibits this issue:
			
			int main() {
			
			std::function<const int&()> F([]{ return 42; });
			
			int x = F(); // oops!
			
			}
			
			Proposed resolution:
			
			Add a second paragraph to the remarks section of 20.14.16.2.1 [func.wrap.func.con]:
			
			template<class F> function(F f);
			
			-7- Requires: F shall be CopyConstructible.
			
			-8- Remarks: This constructor template shall not participate in overload resolution unless

			• F is Lvalue-Callable (20.14.16.2 [func.wrap.func]) for argument types ArgTypes... and return typ and
			• If R is type "reference to T" and INVOKE(ArgTypes...) has value category V and type U:
			  ◦ V is a prvalue, U is a class type, and T is not reference-related (9.4.3 [dcl.init.ref]) to U, and
			  ◦ V is an lvalue or xvalue, and either U is a class type or T is reference-related to U.
			[…]
			
			Tim: LWG in Batavia 2018 would like a way to detect when the initialization of a reference would bi a temporary. This requires compiler support, since there's no known way in the current language to de reliably in the presence of user-defined conversions (see thread starting at https://lists.isocpp.org/lib/2017/07/3256.php).
			
			Meeting:
			Tim wrote p2255 to address this.
			Ville thinks there should be an analysis of alternative approaches.
			Also see P0932.

Core	[CWG794]	Base-derived conversion in member type of pointer-to-member conversion	Related to CWG 170, drafting by Clark seems unlikely. This is section 2.1 of Jeff Snyder's P0149R0, was approved by EWG, 4 years ago, waiting for wording. JF reached out to Jeff. Did wording with Jens, main blocker is lack of implementation. **Meeting:** (notes) Suggest closing as Not A Defect because we have implementation uncertainties, but explore the design space in P0149. ABI group will discuss. All in favor.

Core	[CWG900]	Lifetime of temporaries in range-based for	// some function
			std::vector<int> foo();
			// correct usage
			auto v = foo();
			for(auto i : reverse(v)) { std::cout << i << std::endl; }
			// problematic usage
			for(auto i : reverse(foo())) { std::cout << i << std::endl; }
			Meeting: (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect because it's a c which might have effects on existing code (might cause bugs), and might need to change more than ju range-based loops. See p0614, p0577, p0936, p1179. We'll explore the design space in a separate pap circled back with Nico on this, might write a paper. All in favor.

Core	[CWG1008]	Querying the alignment of an object	https://godbolt.org/z/_SN8iF • GCC already implements this extension without issuing a warning • Clang and EDG implement this extension for gcc compatibility, with a warning • MSVC does not yet implement this feature Quick example using 'auto' illustrates why we might want this capability for objects as well as types. Principle of least astonishment suggests it is surprising for sizeof and alignof to behave differently in regard.

			Recommend shipping this straight to core as soon as we can find a wording champion. We need to discuss with WG14. Questions for EWG to answer before forwarding: • Should this be a unary operator, like sizeof, so "alignof x" is valid? Or should it be like typeid, w parens are required? • What would this mean? Is it the alignment of the type of the object? Or the compiler's best guess alignment of the expression itself? Should it take into account any facts that are known about the expression other than its type? If so, which ones? (For example, if applied to an expression x or x is declared with an `alignas` attribute, is that value returned?) This needs a design paper rather than going straight to core. https://godbolt.org/z/TeVA9T GCC seems to report the alignment of the object not just of decltype(ob **Meeting:** (notes, also discussed in Rapperswil 2014) Suggest closing as Not A Defect, the design is c especially around alignment of object versus type. Invite a paper, Inbal will pitch in, Alidair can colla All in favor. Inbal's paper: P2152R0
Core	[CWG1077]	Explicit specializations in non-containing namespaces	The current wording of 9.8.1.2 [namespace.memdef] and 13.9.3 [temp.expl.spec] requires that an exp specialization be declared either in the same namespace as the template or in an enclosing namespace would be convenient to relax that requirement and allow the specialization to be declared in a non-en namespace to which one or more if the template arguments belongs. Additional note, April, 2015: See EWG issue 48. Might allow us to revert DR2061 and all the horribleness that created and described in p1701. The pr 1077 addresses is the motivating factor in dr2061. Also see CWG 2370. **Meeting:** (notes) Suggest closing as Not A Defect. See p0665, minutes. Continue under p0665 or a fo version of it. All in favor.

Core	[CWG1433]	*trailing-return-type* and point of declaration	template <class T> T list(T x);
			
			template <class H, class ...T>
			
			auto list(H h, T ...args) -> decltype(list(args...)); // list isn't in scope in its own *trailing-return-type*
			
			auto list3 = list(1, 2, 3);
			
			Meeting: (notes, also discussed in Rapperswil 2014) there might be compiler divergence according to Daveed. Suggest closing as Not A Defect, it would be tricky to change behavior without ambiguity. "I this would break existing code that relies on seeing only previous declarations. No objection to unani consent.

Core	[CWG1469]	Omitted bound in array *new-expression*	The syntax for noptr-new-declarator in 7.6.2.7 [expr.new] paragraph 1 requires an expression, even th the bound could be inferred from a braced-init-list initializer. It is not clear whether 9.4.1 [dcl.init.agg paragraph 4, An array of unknown size initialized with a brace-enclosed initializer-list containing n in clauses, where n shall be greater than zero, is defined as having n elements (9.3.3.4 [dcl.array]). should be considered to apply to the new-type-id variant, e.g., new (int[]){1, 2, 3} This was addressed by p1009 **Meeting:** (notes) Suggest closing as Not A Defect. Addressed by P1009. No objection to unanimous c

Core	[CWG1555]	Language linkage and function type compatibility	Currently function types with different language linkage are not compatible, and 7.6.1.2 [expr.call] pa 1 makes it undefined behavior to call a function via a type with a different language linkage. These fe are generally not enforced by most current implementations (although some do) between functions w and C++ language linkage. Should these restrictions be relaxed, perhaps as conditionally-supported b Somewhat related to CWG1463. **Meeting:** (notes) no strong consensus at the moment, Erich Keane brought this up with SG12 Undefi Behavior. Long discussion, will need to revisit, need a paper. **Meeting Oct 22nd 2020:** will need a volunteer to write a paper, but the wording in the standard is unambiguous, and we have existence proof of platforms which use different calling conventions betw and C++. However many major compilers have chosen to ignore this. Not a defect, but we welcome a to change the status quo. No objection to unanimous consent.
Core	[CWG1643]	Default arguments for template parameter packs	Although 13.2 [temp.param] paragraph 9 forbids default arguments for template parameter packs, allo them would make some program patterns easier to write. Should this restriction be removed? **Meeting:** (notes) Suggest closing as Not A Defect. Interesting design space, but needs a paper, see N3 objection to unanimous consent.
Core	[CWG1864]	List-initialization of array objects	The resolution of issue 1467 now allows for initialization of aggregate classes from an object of the s type. Similar treatment should be afforded to array aggregates. See recent discussion on allowing 'auto x[] = {1, 2, 3};' -- this topic came up there. The two questions answered independently, but some consider them to be related. **Meeting:** (notes) Suggest closing as Not A Defect. Fixing anything in this space would require a pape considers the entire design space, Timur might be interested in this. No objection to unanimous conse
Core	[CWG1871]	Non-identifier characters in *ud-suffix*	JF forwarded to SG16 Unicode given their work on TR31. Tracked on GitHub. SG16 reviewed D194 still needs wording changes. Discussed at the April 22nd SG16 telecon. **SG16 Poll:** Is there any objection to unanimous consent for recommending rejection of this proposal? objection to unanimous consent. **EWG Meeting:** (notes 2020-04-15, notes 2020-05-07) Suggest closing as Not A Defect. No objectio unanimous consent.
Core	[CWG1876]	Preventing explicit specialization	A desire has been expressed for a mechanism to prevent explicitly specializing a given class template particular std::initializer_list and perhaps some others in the standard library. It is not clear whether si adding a prohibition to the description of the templates in the library clauses would be sufficient or w core language mechanism is required. **Meeting:** (notes) Suggest closing as Not A Defect. No objection to unanimous consent. This could be language feature, would need library usecase examples. **Poll:** we are interested in such a paper SF 2 F 13 N 6 A 1 SA 0
Core	[CWG1915]	Potentially-invoked destructors in non-throwing constructors	Since the base class constructor is non-throwing, the deleted base class destructor need not be referen There's a typo in the issues list here: it should say "Since the *derived* class constructor is non-throwin deleted base class destructor need not be referenced." **Meeting:** (notes 2020-04-23) the proposed wording changes the implementation leeway in two-phase unwinding, which breaks some existing ABIs. We would need a paper to explore this further. Sugges as Not A Defect. No objection to unanimous consent.

Core	[CWG1923]	Lvalues of type void	There does not seem to be any significant technical obstacle to allowing a void* pointer to be derefer and that would avoid having to use weighty circumlocutions when casting to a reference to an object designated by such a pointer. Might consider this a duplicate of the discussion on "regular void". JF reached out to Matt. He has an implementation in clang. Needs to update the paper. Might need a v to present. Daveed would be interested in presenting. **Meeting:** (notes 2020-04-23) Suggest closing as Not A Defect. Explore under the “regular void” umb No objection to unanimous consent.
Core	[CWG1934]	Relaxing *exception-specification* compatibility requirements	According to 14.5 [except.spec] paragraph 4, If any declaration of a function has an exception-specification that is not a noexcept-specification allo exceptions, all declarations, including the definition and any explicit specialization, of that function sl have a compatible exception-specification. This seems excessive for explicit specializations, considering that paragraph 6 applies a looser require for virtual functions: If a virtual function has an exception-specification, all declarations, including the definition, of any fu that overrides that virtual function in any derived class shall only allow exceptions that are allowed by exception-specification of the base class virtual function. The rule in paragraph 3 is also problematic in regard to explicit specializations of destructors and defa special member functions, as the implicit exception-specification of the template member function ca determined. There is also a related problem with defaulted special member functions and exception-specifications According to 9.5.2 [dcl.fct.def.default] paragraph 3, If a function that is explicitly defaulted has an explicit exception-specification that is not compatible ([except.spec]) with the exception-specification on the implicit declaration, then • if the function is explicitly defaulted on its first declaration, it is defined as deleted; • otherwise, the program is ill-formed. This rule precludes defaulting a virtual base class destructor or copy/move functions if the derived cla function will throw an exception not allowed by the implicit base class member function. **Meeting:** (notes 2020-04-23) JF will work with Mike to update wording to C++20. Will revisit. From Mike: This issue is NAD since we eliminated typed exception-specifications. The current word dealing only with noexcept(true) and noexcept(false), does not have this issue. Will remove “extensio status. **Meeting:** no objection to CWG taking it back, and marking it as NAD.

Core	[CWG1957]	decltype(auto) with direct-list-initialization	Paper N3922 changed the rules for deduction from a braced-init-list containing a single expression in initialization context. Should a corresponding change be made for decltype(auto)? E.g.,
			auto x8a = { 1 }; // decltype(x8a) is std::initializer_list<int>
			decltype(auto) x8d = { 1 }; // ill-formed, a braced-init-list is not an expression
			auto x9a{ 1 }; // decltype(x9a) is int
			decltype(auto) x9d{ 1 }; // decltype(x9d) is int
			See also issue 1467, which also effectively ignores braces around a single expression, this change wo parallel to that one, even though the primary motivation for delctype(auto) is in the return type of a forwarding function, where direct-initialization does not apply.
			
			Meeting: (notes 2020-04-23) Suggest closing as Not A Defect. This would be a language change, it's that we want to make such a change. It would require a paper. Mike Spertus is writing a paper with sc overlap, will cover this as well. No objection to unanimous consent.

Core	[CWG2111]	Array temporaries in reference binding	The current wording of the Standard appears to permit code like
			`void f(const char (&)[10]);`
			`void g() {`
			`  f("123");`
			`  f({'a','b','c','\0'});`
			`}`
			creating a temporary array of ten elements and binding the parameter reference to it. This is controver should be reconsidered. (See issues 1058 and 1232.)
			
			Meeting: (notes 2020-04-23) JF digging more, talking to Richard about this.
			Somewhat related to P2174 compound literals.
			Would be very strange if
			`const char (&&x)[10] = {'a', 'b', 'c'};`
			... were invalid but ...
			`const char x[10] = {'a', 'b', 'c'};`
			`const char (&&x)[3] = {'a', 'b', 'c'};`
			... were both OK. Maybe either we should allow the trailing elements to be zeroed in general (the stat or not (a major breaking change). Which means we should reject the issue on consistency grounds. Tl general rule is that if
			T &&r = init;
			... can't bind directly, we create a temporary initialized as if with
			T r = init;
			... and bind to that. (And similarly for const references.)
			Another question: Do we want to support (T){inits} as a synonym for T{inits}?
			
			Meeting Oct 22nd 2020: Note that the issue itself is defective, and `f("123")` isn't valid.

			CWG2352 references p1358, and was resolved, leading us to believe that this is now mainstream and controversial. Not a defect. No objection to unanimous consent.

Core	[CWG2125]	Copy elision and comma operator	Currently, _N4750_.15.8 [class.copy] paragraphs 31-32 apply only to the name of a local variable in determining whether a return expression is a candidate for copy elision or move construction. Would sense to extend that to include the right operand of a comma operator?
			
			X f() {
			X x;
			return (0, x);
			}
			
			Meeting: (notes 2020-04-23) Consider expanding to other places that expand bit-field-ness such as re throw : x;. Suggest closing as Not A Defect. Will need a paper to address, no current volunteer for thi objection to unanimous consent.

Core	[CWG2132]	Deprecated default generated copy constructors	EWG has indicated that they are not currently in favor of removing the implicitly declared defaulted c constructors and assignment operators that are eprecated in _N4750_.15.8 [class.copy] paragraphs 7 a Should this deprecation be removed? Related: discussing under p2139 deprecations. **Meeting:** (note 2020-04-29) Suggest closing as Not A Defect. No objection to unanimous consent. W want to remove entirely, or un-deprecate. This will need a paper, Ville will talk with Alisdair about fo the topic from p2139.

Core	[CWG914]	Value-initialization of array types	Although value-initialization is defined for array types and the () initializer is permitted in a mem-init naming an array member of a class, the syntax T() (where is an array type) is explicitly forbidden by [expr.type.conv] paragraph 2. This is inconsistent and the syntax should be permitted.
			
			Rationale (July, 2009): The CWG was not convinced of the utility of this extension, especially in ligh questions about handling the lifetime of temporary arrays. This suggestion needs a proposal and analy the EWG before it can be considered by the CWG.
			
			This has become a more severe inconsistency after we adopted Ville's P0960 for C++20. Now it's no that has this weird special-case restriction, it's (a, b, c) too:
			
			using X = int[];
			
			X x{1, 2, 3}; // ok, int[3]
			
			X y(1, 2, 3); // ok, int[3]
			
			f(X{1, 2, 3}); // ok, int[3] temporary
			
			f(X(1, 2, 3)); // error
			
			Meeting: (notes) it is a defect, need a paper. David Stone trying to find a volunteer to write said pape favor.

Core	[CWG1463]	extern "C" alias templates	Currently 13 [temp] paragraph 4 forbids any template from having C linkage. Should alias templates exempt from this prohibition, since they do not have any linkage? Additional note, April, 2013: It was suggested (see messages 23364 through 23367) that relaxing this restriction for alias templates could provide a way of addressing the long-standing lack of a way of sp

			a language linkage for a dependent function type (see issue 13). The priority was deleted to allow CW consider the implications of that potential application of the facility. We should either have some way to express a dependent function type with C language linkage (and s accept 1555 below) or we should remove the notion that C language linkage (or not) is part of the fun type at all. (Apparently some targets used it; are they still in use? EDG probably knows.) Actively discussed on CWG reflector. **Davis' interpretation of CWG discussion**: I don't speak for Core, of course, but my (Evolutionary) thoughts are those given (last) in the messag which you replied: neither the wording nor the apparent intent of [temp.pre]/6's restrictions on C link templates make any sense. Since templates (and explicit specializations) can't have C linkage of the mangling variety (which the standard calls language linkage of a (function) name) but can have C linl the calling-convention variety (which the standard calls language linkage of a (function) type), extern should grant them the latter and not the former with no error. This has certain obvious applications in C APIs with callbacks. (Put differently, CWG13 shouldn't have been rejected; it appears to have gott bogged down in questions of syntax, but we have an adequate syntactic workaround that we merely h permit.) CWG1463 asks for a (very) proper subset of the above: merely that extern "C" be allowed to apply to templates (claiming incorrectly that they have no name with linkage at all). I consider it more relevar more productive) to talk about whether entities have names with language linkage than with the ordin kind. The Core reflector discussion (which seems to have finished for the moment) also touched on the cas "the same" function template is declared with two different language linkages for its type, but the onl relevant Evolution input there would be a decision to go the opposite way and forbid function templat having types with C language linkage entirely. (They currently can, but it's mostly or entirely useless If (for CWG1555, also on your list) the rules about language linkage of (function) types are sufficient relaxed, then CWG1463 may be moot (and my extension of it with it), but that seems unlikely given l recent identification of a case where they matter. **Meeting:** (notes 2020-04-15, notes 2020-10-22, also discussed in Rapperswil 2014) We agree that thi issue. extern "C" on a template should only affect calling convention, and not the mangling. Davis an Hubert volunteer to write a paper. Unanimous consent.
Core	[CWG1790]	Ellipsis following function parameter pack	Although the current wording permits an ellipsis to immediately follow a function parameter pack, it clear that the <cstdarg> facilities permit access to the ellipsis arguments. The problem here (which is not explained in the issue) is: how do you supply the name of the last par before the ellipsis to va_start? You can't put the name of a pack there (it wouldn't be expanded) and t no way to name the last element of the pack (nor to deal with the case where the pack is empty). **Meeting:** (notes) 3 options: fix wording around "last parameter", remove facility entirely (either va_s function declarator), try to invent a language facility. JF emailed EWG to see if anyone has a strong preference, or if we should send back to CWG to fix wording, long discussion. **Michael Spertus:** I am willing to commit to including and analysis on this in an upcoming paper on parameter packs. JF followed up with Michael and Barry, no response.
Core	[CWG1962]	Type of __func__	Two questions have arisen regarding the treatment of the type of the __func__ built-in variable. First, implementations accept void f() {

			typedef decltype(__func__) T;
			
			T x = __func__;
			
			}
			
			even though T is specified to be an array type.
			
			In a related question, it was noted that __func__ is implicitly required to be unique in each function, a not only the value but the type of __func__ are implementation-defined; e.g., in something like
			
			inline auto f() { return &__func__; }
			
			the function type is implementation-specific. These concerns could be addressed by making the value prvalue of type const char* instead of an array lvalue.
			
			Rationale (November, 2018): See also issue 2362, which asks for the ability to use __func__ in a con function. These two goals are incompatible.
			
			The deep question here is about __func__ and the ODR. Does EWG want implementations to someh behave as if __func__ is the same in all copies of an inline function (in which case it can have an arra and be usable in constant expressions, but the *demangling* algorithm used to construct it becomes par ABI), or does EWG want implementations to behave as if __func__ may differ between copies, so is not known until runtime (in which case it must have either pointer or incomplete array type, and its va not usable in constant expressions -- but its address could still be usable).
			
			Note: C++20 has std::source_location::function_name.
			
			Meeting: (notes 2020-04-23) We should discuss this with WG14. This is indeed a language issue. No objection to unanimous consent. We'll need a paper, potentially considering what Reflection could do brought it up on the mailing list.
			
			We probably need a paper to disentangle this.

Core	[CWG2228]	Ambiguity resolution for cast to function type	Proposed resolution. C++ has a blanket disambiguation rule that says if a sequence of tokens can be interpreted as either a name or an expression, the type-name interpretation is chosen. This is unhelpful in cases like (T())+3 where the current rules make "(T())" a cast to the type "function with no parameters returning T" rath parenthesized value-initialization of a temporary of type T. The former interpretation is always ill-for you can't cast to a function type - while the latter could be useful. Richard's proposed resolution, cited above, is to avoid the ambiguity by changing the grammar so tha like "(T())" cannot be parsed as a cast. The wording in the proposal applies that change to a number o different contexts where the ambiguity can come into play. There are two contexts where the change i applied, however: 1) the operand of typeid, and 2) a template-argument. During our discussion yester there was some support for the idea of applying the change to typeid, as well, although that would be breaking change for any programs that rely on the current disambiguation to get type information for function types. There was general agreement, however, to exclude template arguments, since they are for things like std::function.

			CWG would, therefore, like some guidance from EWG on two questions. First, should we apply the r syntax to the operand of typeid, even though it's a breaking change? More generally, the question was raised whether we should make this change at all. Although resolvi ambiguity in the other direction would arguably be more convenient in many cases, there is a tension that convenience and the complexity of the language. In particular, we would be creating a situation v the exact same sequence of tokens would mean two different things, depending on the context in whi appear. Is the convenience worth the cost in complexity? **Meeting:** (note 2020-04-29, notes 2020-03-23, note from Core summer 2020, notes from Core Pragu notes from Core 2019-01-07, notes from Core Kona 2019, notes from Core Cologne 2019) This is an we'd like to change the standard to resolve it: SF 0 F 6 N 4 A 3 SA 2 Davis emailed EWG reflector. Long discussion.

Core	[CWG2296]	Are default argument instantiation failures in the "immediate context"?	Example 1:
			
			template <typename T, typename U = T>
			
			void fun(T v, U u = U()) {}
			
			void fun(...) {}
			
			struct X {
			
			X(int) {} // no default ctor
			
			};
			
			int main() { fun (X(1)); }
			
			Consider the following example (taken from issue 3 of paper P0348R0):
			
			Example 2:
			
			template <typename U> void fun(U u = U());
			
			struct X {
			
			X(int) {}
			
			};
			
			template <class T> decltype(fun<T>()) g(int) { }
			
			template <class> void g(long) { }
			
			int main() { g<X>(0); }
			
			When is the substitution into the return type done? The current specification makes this example ill-f because the failure to instantiate the default argument in the decltype operand is not in the immediate of the substitution, although a plausible argument for making this a SFINAE case can be made.
			
			Meeting: (notes 2020-05-07)
			
			The first example under issue 3 of paper P0348R0 should become well-formed.
			
			SF F N A SA
			
			0 1 4 3 6

			The second example under issue 3 of paper P0348R0 should become well-formed. SF F N A SA 1 5 7 3 0 This is an issue. We'd like to see a paper addressing it. It should explore what "Immediate context" m No objection to unanimous consent. Daveed / Hubert might entertain writing a paper explaining this. Ville emailed EWG about this. JF contacted Andrzej to see if he's interested in addressing this. Not sure he is. **Meeting 2020-10-28**: Tomasz will write a paper which exposes the issue and what he thinks should b (but not resolving wording itself). Hubert remembers an email about this, will find it and sync with JI **Meeting 2021-01-14:** Andrzej wrote a paper for this (emailed 2021-01-12 to EWG).

Core	[CWG2362]	__func__ should be constexpr	The definition of __func__ in 9.5.1 [dcl.fct.def.general] paragraph 8 is:
			
			static const char __func__[] = "function-name";
			
			This prohibits its use in constant expressions, e.g.,
			
			int main () {
			
			// error: the value of __func__ is not usable in a constant expression
			
			constexpr char c = __func__[0];
			
			}
			
			Rationale (November, 2018): See also issue 1962, which asks that the type of __func__ be const char
			two goals are incompatible.
			
			Meeting: handle with 1962.

§ 9. Near-future EWG plans

We will continue to work on issue resolution and C++23, prioritizing according to [P0592R4].

§ References

§ Informative References

[CWG1008]
Steve Clamage. Querying the alignment of an object. 27 November 2009. extension. URL: https://wg21.link/cwg1008

[CWG1077]
Mike Spertus. Explicit specializations in non-containing namespaces. 13 June 2010. extension. URL: https://wg21.link/cwg1077

[CWG1433]
Jason Merrill. trailing-return-type and point of declaration. 20 December 2011. extension. URL: https://wg21.link/cwg1433

[CWG1463]
Daveed Vandevoorde. extern "C" alias templates. 19 August 2011. extension. URL: https://wg21.link/cwg1463

[CWG1469]
Johannes Schaub. Omitted bound in array new-expression. 12 February 2012. extension. URL: https://wg21.link/cwg1469

[CWG1555]
Daniel Krügler. Language linkage and function type compatibility. 18 September 2012. extension. URL: https://wg21.link/cwg1555

[CWG1643]
Vinny Romano. Default arguments for template parameter packs. 17 March 2013. extension. URL: https://wg21.link/cwg1643

[CWG1790]
Daryle Walker. Ellipsis following function parameter pack. 1 October 2013. extension. URL: https://wg21.link/cwg1790

[CWG1864]
Alisdair Meredith. List-initialization of array objects. 15 February 2014. extension. URL: https://wg21.link/cwg1864

[CWG1871]
Richard Smith. Non-identifier characters in ud-suffix. 17 February 2014. extension. URL: https://wg21.link/cwg1871

[CWG1876]
John Spicer. Preventing explicit specialization. 19 February 2014. extension. URL: https://wg21.link/cwg1876

[CWG1915]
Aaron Ballman. Potentially-invoked destructors in non-throwing constructors. 15 April 2014. extension. URL: https://wg21.link/cwg1915

[CWG1923]
Richard Smith. Lvalues of type void. 6 May 2014. extension. URL: https://wg21.link/cwg1923

[CWG1934]
Vinny Romano. Relaxing exception-specification compatibility requirements. 3 June 2014. extension. URL: https://wg21.link/cwg1934

[CWG1957]
Vinny Romano. decltype(auto) with direct-list-initialization. 20140630. extension. URL: https://wg21.link/cwg1957

[CWG1962]
Steve Clamage. Type of __func__. 4 July 2014. extension. URL: https://wg21.link/cwg1962

[CWG2111]
Vinny Romano. Array temporaries in reference binding. 2 April 2015. extension. URL: https://wg21.link/cwg2111

[CWG2125]
Vinny Romano. Copy elision and comma operator. 6 May 2015. extension. URL: https://wg21.link/cwg2125

[CWG2132]
Nico Josuttis. Deprecated default generated copy constructors. 28 May 2015. extension. URL: https://wg21.link/cwg2132

[CWG2228]
Richard Smith. Ambiguity resolution for cast to function type. 2 February 2016. review. URL: https://wg21.link/cwg2228

[CWG2261]
Jens Maurer. Explicit instantiation of in-class friend definition. 28 April 2016. extension. URL: https://wg21.link/cwg2261

[CWG2270]
Richard Smith. Non-inline functions and explicit instantiation declarations. 10 June 2016. extension. URL: https://wg21.link/cwg2270

[CWG2296]
Jason Merrill. Are default argument instantiation failures in the “immediate context”?. 25 June 2016. extension. URL: https://wg21.link/cwg2296

[CWG2362]
Anthony Polukhin. __func__ should be constexpr. 23 October 2017. extension. URL: https://wg21.link/cwg2362

[CWG794]
CH. Base-derived conversion in member type of pointer-to-member conversion. 3 March 2009. extension. URL: https://wg21.link/cwg794

[CWG900]
Thomas J. Gritzan. Lifetime of temporaries in range-based for. 12 May 2009. extension. URL: https://wg21.link/cwg900

[CWG914]
Gabriel Dos Reis. Value-initialization of array types. 10 June 2009. extension. URL: https://wg21.link/cwg914

[LWG2432]
David Krauss. initializer_list assignability. EWG. URL: https://wg21.link/lwg2432

[LWG2813]
Brian Bi. std::function should not return dangling references. EWG. URL: https://wg21.link/lwg2813

[N4514]
Michael Wong. Technical Specification for C++ Extensions for Transactional Memory. 8 May 2015. URL: https://wg21.link/n4514

[P0592R4]
Ville Voutilainen. To boldly suggest an overall plan for C++23. 25 November 2019. URL: https://wg21.link/p0592r4

[P1000R4]
Herb Sutter. C++ IS schedule. 14 February 2020. URL: https://wg21.link/p1000r4

[P1018r11]
JF Bastien. C++ Language Evolution status - pandemic edition - 2021/05. 1 June 2021. URL: https://wg21.link/p1018r11

[P1018r9]
JF Bastien. C++ Language Evolution status - pandemic edition - 2021/01–2021/03. 8 March 2021. URL: https://wg21.link/p1018r9

[P1371R3]
Michael Park, Bruno Cardoso Lopes, Sergei Murzin, David Sankel, Dan Sarginson, Bjarne Stroustrup. Pattern Matching. 15 September 2020. URL: https://wg21.link/p1371r3

[P1393R0]
D. S. Hollman, Chris Kohlhoff, Bryce Lelbach, Jared Hoberock, Gordon Brown, Michał Dominiak. A General Property Customization Mechanism. 13 January 2019. URL: https://wg21.link/p1393r0

[P1401R3]
Andrzej Krzemieński. Narrowing contextual conversions to bool. 27 May 2020. URL: https://wg21.link/p1401r3

[P1467R4]
David Olsen, Michał Dominiak. Extended floating-point types and standard names. 14 June 2020. URL: https://wg21.link/p1467r4

[P1701R1]
Nathan Sidwell. Inline Namespaces: Fragility Bites. 13 September 2020. URL: https://wg21.link/p1701r1

[P1726R3]
Paul E. McKenney, Maged Michael, Jens Mauer, Peter Sewell, Martin Uecker, Hans Boehm, Hubert Tong, Niall Douglas, Will Deacon, Michael Wong, and David Goldblatt. Pointer lifetime-end zap. 21 February 2020. URL: https://wg21.link/p1726r3

[P1858R2]
Barry Revzin. Generalized pack declaration and usage. 1 March 2020. URL: https://wg21.link/p1858r2

[P1875R0]
Michael Spear, Hans Boehm, Victor Luchangco, Michael Scott, Michael Wong. Transactional Memory Lite Support in C++. 7 October 2019. URL: https://wg21.link/p1875r0

[P1875r2]
Michael Spear, Hans Boehm, Victor Luchangco, Michael Scott, Michael Wong. Transactional Memory Lite Support in C++. 15 March 2021. URL: https://wg21.link/p1875r2

[P1938R1]
Barry Revzin, Daveed Vandevoorde, Richard Smith, Andrew Sutton. if consteval. 2 March 2020. URL: https://wg21.link/p1938r1

[P1949R4]
Steve Downey, Zach Laine, Tom Honermann, Peter Bindels, Jens Maurer. C++ Identifier Syntax using Unicode Standard Annex 31. 5 June 2020. URL: https://wg21.link/p1949r4

[P1949R6]
Steve Downey, Zach Laine, Tom Honermann, Peter Bindels, Jens Maurer. C++ Identifier Syntax using Unicode Standard Annex 31. 15 September 2020. URL: https://wg21.link/p1949r6

[P1967R2]
JeanHeyd Meneide. #embed - a simple, scannable preprocessor-based resource acquisition method. 2 March 2020. URL: https://wg21.link/p1967r2

[P1999R0]
Ville Voutilainen. Process proposal: double-check evolutionary material via a Tentatively Ready status. 25 November 2019. URL: https://wg21.link/p1999r0

[P2011R1]
Barry Revzin, Colby Pike. A pipeline-rewrite operator. 16 April 2020. URL: https://wg21.link/p2011r1

[P2013R2]
Ben Craig. Freestanding Language: Optional ::operator new. 14 August 2020. URL: https://wg21.link/p2013r2

[P2025R1]
Anton Zhilin. Guaranteed copy elision for return variables. 15 June 2020. URL: https://wg21.link/p2025r1

[P2036r2]
Barry Revzin. Changing scope for lambda trailing-return-type. 23 July 2021. URL: https://wg21.link/p2036r2

[P2044R0]
Robert Leahy. Member Templates for Local Classes. 12 January 2020. URL: https://wg21.link/p2044r0

[P2066r7]
Jens Maurer, Hans Boehm, Victor Luchangco, Michael L. Scott, Michael Spear, and Michael Wong. Suggested draft TS for C++ Extensions for Minimal Transactional Memory. 14 May 2021. URL: https://wg21.link/p2066r7

[P2128r5]
Corentin Jabot, Isabella Muerte, Daisy Hollman, Christian Trott, Mark Hoemmen. Multidimensional subscript operator. 13 April 2021. URL: https://wg21.link/p2128r5

[P2138r4]
Ville Voutilainen. Rules of Design<=>Specification engagement. 19 April 2021. URL: https://wg21.link/p2138r4

[P2139R1]
Alisdair Meredith. Reviewing Deprecated Facilities of C++20 for C++23. 15 June 2020. URL: https://wg21.link/p2139r1

[P2139R2]
Alisdair Meredith. Reviewing Deprecated Facilities of C++20 for C++23. 15 July 2020. URL: https://wg21.link/p2139r2

[P2145R1]
Bryce Adelstein Lelbach, Titus Winters, Fabio Fracassi, Billy Baker, Nevin Liber, JF Bastien, David Stone, Botond Ballo, Tom Honermann. Evolving C++ Remotely. 15 September 2020. URL: https://wg21.link/p2145r1

[P2156R0]
Erich Keane. Allow Duplicate Attributes. 15 April 2020. URL: https://wg21.link/p2156r0

[P2173R0]
Daveed Vandevoorde, Inbal Levi, Ville Voutilainen. Attributes on Lambda-Expressions. 15 May 2020. URL: https://wg21.link/p2173r0

[P2174R0]
Zhihao Yuan. Compound Literals. 16 May 2020. URL: https://wg21.link/p2174r0

[P2186R0]
JF Bastien, Alisdair Meredith. Removing Garbage Collection Support. 12 July 2020. URL: https://wg21.link/p2186r0

[P2195R0]
Bryce Adelstein Lelbach. Electronic Straw Polls. 15 September 2020. URL: https://wg21.link/p2195r0

[P2201R0]
Jens Maurer. Mixed string literal concatenation. 14 July 2020. URL: https://wg21.link/p2201r0

[P2223R0]
Corentin Jabot. Trimming whitespaces before line splicing. 14 September 2020. URL: https://wg21.link/p2223r0

[P2246r1]
Aaron Ballman. Character encoding of diagnostic text. 15 January 2021. URL: https://wg21.link/p2246r1

[P2266r1]
Arthur O'Dwyer. Simpler implicit move. 14 March 2021. URL: https://wg21.link/p2266r1

[P2314r2]
Jens Maurer. Character sets and encodings. 14 May 2021. URL: https://wg21.link/p2314r2

[P2316r1]
Corentin Jabot. Consistent character literal encoding. 11 July 2021. URL: https://wg21.link/p2316r1

[P2334r1]

Melanie Blower. Add support for preprocessing directives elifdef and elifndef. 20210430. URL: https://wg21.link/p2334r1

[P2360r0]

Jens Maurer. Extend init-statement to allow alias-declaration. 13 April 2021. URL: https://wg21.link/p2360r0